

- **Modello Dati JSON per Oltre 100 Plugin Supportati**

"Un database. Ogni plugin. Nessuna eccezione."

Importanza: ★★★★★ — Fondamentale per il funzionamento del sistema

- **1. Panorámica del Database Plugin**

```

    PLUGIN DATABASE

    plugins.json (versione 2.0, ~5MB per 100 plugin)

    pluginEntry[]
    ├── metadata (vendor, version, formati)
    ├── parameters[] (fino a 500 per plugin)
    ├── presets[] (fino a 300 factory preset)
    ├── knownProcedures[] (configurazioni AI)
    └── mappings[] (VST3 GUID, MIDI CC, OSC)

    Git: plugins.json → versionato, review PR obbligatoria
    CI: validazione JSON Schema + test integrità

```

```
{
  "schemaVersion": "2.0",
  "lastUpdated": "2026-05-12",
  "pluginCount": 100,
  "plugins": [
    { /* PluginEntry */ },
    { /* PluginEntry */ }
  ],
  "categories": { /* CategoryDefinition */ },
  "$schema": "../schema/plugin-database.schema.json"
}
```

• 2. Schema Principale Plugin — PluginEntry

Ogni plugin è rappresentato da un oggetto `PluginEntry` con la seguente struttura:

```
{
  "id": "fabfilter_proq3",
  "name": "Pro-Q 3",
  "vendor": "FabFilter",
  "version": "3.22",
  "category": "ParametricEQ",
  "subcategory": "DynamicEQ",
  "formats": ["VST3", "AU", "AAX"],
  "vst3Guid": "E8E38A93-21B0-4B8D-B7EC-B54B8B9CF48C",
  "auComponentId": "FFEQ",
  "aaxId": "com.fabfilter.ProQ3",
  "parameters": [ /* ParameterEntry[] */ ],
  "presets": [ /* PresetEntry[] */ ],
  "knownProcedures": [ /* ProcedureEntry[] */ ],
  "metadata": {
    "tags": ["eq", "dynamic-eq", "linear-phase", "spectrum"],
    "imageUrl": "https://cdn.fabfilter.com/img/pro-q3.png",
    "manualUrl": "https://www.fabfilter.com/help/pro-q3",
    "minVersion": "3.0",
    "testedVersion": "3.22",
    "releaseDate": "2024-09-15",
    "categoryPath": ["EQ", "Parametric", "Dynamic"]
  }
}
```

CAMPO	TIPO	DESCRIZIONE	OBBLIGATORIO
<code>id</code>	string	Identificatore unico snake_case	✓
<code>name</code>	string	Nome commerciale del plugin	✓
<code>vendor</code>	string	Sviluppatore / azienda	✓
<code>version</code>	string	Versione testata	✓
<code>category</code>	string	Categoria principale (vedi §4)	✓
<code>formats</code>	string[]	Formati supportati	✓
<code>parameters</code>	array	Lista parametri esposti	✓
<code>metadata.tags</code>	string[]	Tag per ricerca semantica	✗
<code>metadata.manualUrl</code>	string	URL manuale utente	✗
<code>metadata.minVersion</code>	string	Versione minima richiesta	✗

[NOTE] Il campo `vst3Guid` è critico per il riconoscimento automatico del plugin quando viene caricato in un host VST3. Senza un GUID corretto, WhyCremisi non può identificare il plugin in modo deterministico.

• 3. Schema Parametri — `ParameterEntry`

Il cuore del database: ogni parametro di ogni plugin è definito con la massima precisione.

— 3.1 STRUTTURA COMPLETA

```
{
  "id": "band1_frequency",
  "name": "Band 1 Frequency",
  "index": 0,
  "type": "float",
  "defaultValue": 1000.0,
  "min": 20.0,
  "max": 20000.0,
  "step": 0.1,
  "unit": "Hz",
  "group": "Band 1",
  "hidden": false,
  "automationRate": "continuous",
  "mapping": {
    "midiCC": 74,
    "oscPath": "/plugin/fabfilter/proq3/band1/freq"
  },
  "aliases": ["freq1", "frequency band 1", "banda 1 frequenza"],
  "scaling": {
    "type": "logarithmic",
    "normalizedMin": 0.0,
    "normalizedMax": 1.0,
    "physicalMin": 20.0,
    "physicalMax": 20000.0
  }
}
```

— 3.2 TIPI DI PARAMETRO

TIPO	ESEMPI	RANGE
float	Gain, Frequency	min..max (continuo)
int	Bands, Oversampling	min..max (discreto)
bool	Bypass, Solo, Mute	0 / 1
enum	Filter Type, Style	lista di stringhe
text	Preset Name	stringa libera

— 3.3 GRUPPI DI PARAMETRI

Molti plugin moderni raggruppano i parametri in sezioni logiche:

```
{
  "groups": {
    "Band 1": { "index": 0, "params": ["band1_frequency", "band1_gain", "band1_q", "band1_type", "band1_automationRate"], "hidden": false },
    "Band 2": { "index": 1, "params": ["band2_frequency", "band2_gain", "band2_q", "band2_type", "band2_automationRate"], "hidden": false },
    "Output": { "index": 24, "params": ["output_gain", "phase_mode", "bypass"] }
  }
}
```

• 4. Schema Categorie

```
├─ EQ
│   ├── GraphicEQ
│   ├── ParametricEQ
│   ├── DynamicEQ
│   ├── Shelf
│   ├── FilterBank
│   └─ Crossover
├─ Dynamics
│   ├── Compressor
│   │   ├── TubeComp
│   │   ├── BusComp
│   │   ├── MasteringComp
│   │   └─ MultibandComp
│   ├── Limiter
│   │   ├── SoftClip
│   │   ├── BrickwallLimiter
│   │   └─ TruePeakLimiter
│   ├── Gate
│   ├── Expander
│   │   └─ GateExpander
│   ├── TransientShaper
│   └─ Ducker
│       └─ Sidechain
├─ Distortion
│   ├── Clipper
│   ├── Saturator
│   │   ├── TapeSimulator
│   │   └─ BitCrusher
│   ├── Saturation
│   └─ Exciter
├─ Spatial
│   ├── Reverb
│   │   └─ ReverbImpulse
│   ├── Delay
│   ├── Panner
│   ├── Imager
│   └─ StereoField
├─ Modulation
│   ├── Chorus
│   ├── Flanger
│   ├── Phaser
│   ├── Tremolo
│   ├── Vibrato
│   ├── Rotary
│   └─ Wah
├─ Pitch
│   ├── PitchShifter
│   └─ Harmonizer
├─ Spectral
│   ├── SpectralProcessor
│   └─ LoudnessMaximizer
├─ Analyzer
│   ├── Spectrum
│   ├── Loudness
│   └─ PhaseCorrelation
├─ Meter
├─ Utility
└─ Instrument
    ├── Synth
    ├── Sampler
    └─ -
```

— 4.2 MAPPA CATEGORIE IN JSON

```
{
  "categories": {
    "ParametricEQ": {
      "displayName": "Parametric EQ",
      "icon": "eq-parametric",
      "parents": ["EQ"],
      "color": "#4FC3F7"
    },
    "BrickwallLimiter": {
      "displayName": "Brickwall Limiter",
      "icon": "limiter-brickwall",
      "parents": ["Dynamics", "Limiter"],
      "color": "#FF5252"
    },
    "TubeComp": {
      "displayName": "Tube Compressor",
      "icon": "comp-tube",
      "parents": ["Dynamics", "Compressor"],
      "color": "#FFB74D"
    }
  }
}
```

— 4.3 MATRICE DI COMPATIBILITÀ CATEGORIA × FORMATO

CATEGORIA	VST3	AU	AAX	AAX-S	Notes
ParametricEQ	✓	✓	✓	✓	
DynamicEQ	✓	✓	✓	✓	
Compressor	✓	✓	✓	✓	
MultibandComp	✓	✓	✓	✗	AAX-S non supporta
Limiter	✓	✓	✓	✓	
Reverb	✓	✓	✓	✓	
Synth	✓	✓	✗	✗	AAX non supporta strumenti
Sampler	✓	✓	✗	✗	
SpectralProcessor	✓	✓	✓	✗	

• 5. Preset Inclusi

Ogni plugin può avere preset factory predefiniti che WhyCremisi usa come punti di partenza per le elaborazioni AI.

— 5.1 STRUTTURA PRESET

```
{
  "id": "proq3_vocal_clarity",
  "name": "Vocal Clarity",
  "pluginId": "fabfilter_proq3",
  "category": "vocals",
  "params": {
    "band1_type": "HighPass",
    "band1_frequency": 80,
    "band1_q": 0.707,
    "band2_type": "Peak",
    "band2_frequency": 320,
    "band2_gain": -2.5,
    "band2_q": 1.2,
    "band3_type": "Peak",
    "band3_frequency": 3500,
    "band3_gain": 2.0,
    "band3_q": 0.8,
    "output_gain": 0.0
  },
  "tags": ["vocals", "clarity", "presence", "clean"],
  "rating": 4.5,
  "author": "WhyCremisi Factory",
  "description": "High-pass at 80Hz, subtle cut at 320Hz for mud, 3.5kHz presence boost"
}
```

— 5.2 ORGANIZZAZIONE PRESET

plugins.json → presets[]

Per plugin:

- FabFilter Pro-Q3: 25 preset (vocal, kick, bass, ...)
- iZotope Ozone 10: 20 preset (mastering chain, ...)
- ValhallaRoom: 30 preset (hall, plate, chamber, ...)
- Serum: 200+ preset (bass, lead, pad, FX, ...)
- Kontakt 7: 50 preset (orchestral, drums, ...)

Totale stimato: ~4,000+ preset factory

[NOTE] I preset factory sono il punto di partenza per l'AI.
WhyCremisi li usa come base e li modifica dinamicamente in base
al contesto della sessione, alle preferenze utente e all'analisi
FFT del materiale audio.

• 6. Known Procedures

Le procedure note sono sequenze di operazioni che WhyCremisi può eseguire automaticamente su uno o più plugin. Rappresentano il "know-how" codificato del sistema.

```
{
  "id": "vocal_chain_setup",
  "name": "Configure Vocal Chain",
  "description": "Configura una catena di elaborazione vocale completa con EQ, compressione e de-essing",
  "category": "mixing",
  "tags": ["vocals", "chain", "mixing"],
  "steps": [
    {
      "action": "setParameter",
      "target": "plugin",
      "pluginId": "fabfilter_proq3",
      "params": {
        "band1_type": "HighPass",
        "band1_frequency": 80,
        "band1_enabled": true
      },
      "description": "High-pass filter at 80Hz"
    },
    {
      "action": "setParameter",
      "target": "plugin",
      "pluginId": "fabfilter_proc2",
      "params": {
        "style": "Vocal",
        "threshold": -18.0,
        "ratio": 3.0,
        "attack": 0.5,
        "release": 50.0,
        "makeup_gain": 2.0
      },
      "description": "Set compressor for vocals"
    },
    {
      "action": "setParameter",
      "target": "plugin",
      "pluginId": "fabfilter_prods2",
      "params": {
        "frequency": 5000,
        "amount": 3.0,
        "mode": "Split"
      },
      "description": "De-esser at 5kHz"
    },
    {
      "action": "analyze",
      "target": "fft",
      "params": {
        "windowSize": 4096,
        "checkRange": [80, 20000]
      },
      "description": "Verify spectral balance after chain setup"
    }
  ],
  "pluginIds": ["fabfilter_proq3", "fabfilter_proc2", "fabfilter_prods2"],
  "estimatedTimeMs": 45
}
```


— 6.2 PROCEDURE PREDEFINITE (PARZIALE)

PROCEDURA	PLUGIN COINVOLTI	AZIONI
Vocal Chain Setup	Pro-Q3, Pro-C2, Pro-DS2	HPF → Compression → De-ess
Master Bus Chain	Pro-MB, Pro-L2	Multiband → Limiter
Sidechain Setup	Pro-C2 + kick	Key input → Threshold → Ratio
Parallel Compression	Pro-C2 (mix 30%)	Blend wet/dry
Bass Cleanup	Pro-Q3	HPF 40Hz → Cut mud 200-400Hz
Room Taming	ValhallaRoom + Pro-Q3	Reverb → EQ cut sibilance
Stereo Widening	Imager	M/S → Width expansion

— 6.3 DIAGRAMMA DI ESECUZIONE PROCEDURA

Utente: "set up my vocal chain"

WhyCremisi lookup → "vocal_chain_setup"

├ Step 1: Pro-Q3 → HPF @ 80Hz

├ Step 2: Pro-C2 → Threshold -18dB, Ratio 3:1

├ Step 3: Pro-DS2 → De-ess @ 5kHz

└ Step 4: Verify → FFT check

▼

[Proposta all'utente con undo disponibile]

• 7. Mapping DAW ↔ Plugin

Il riconoscimento plugin è il meccanismo con cui WhyCremisi identifica quale plugin è caricato su quale traccia.

— 7.1 STRATEGIA DI RICONOSCIMENTO A 3 LIVELLI

LIVELLO 1 – GUID Matching (VST3)

```
| Plugin caricato in DAW → chiede VST3 GUID |  
| GUID = "E8E38A93-21B0-4B8D-B7EC-B54B8B9..." |  
|           ↓ |  
| Match su plugins.json[].vst3Guid |  
| Trovato → FabFilter Pro-Q3 |
```

LIVELLO 2 – Fuzzy Match (Fallback)

```
| Plugin senza GUID / formato sconosciuto |  
| → Match per vendor + name |  
| → Algoritmo Levenshtein distance |  
|   vendor:"FabFilter" + name:"Pro-Q 3" |  
| → plugins.json[].vendor + .name |
```

LIVELLO 3 – Feature Detection (Ultima ratio)

```
| Plugin sconosciuto ma con parametri esposti |  
| → Analisi pattern parametri |  
| → Confronto con firme note nel DB |  
| → "24 bande parametriche + spectrum" → EQ |
```

— 7.2 STRUTTURA MAPPING

```
{  
  "mappings": {  
    "vst3Guid": { "type": "exact", "field": "vst3Guid" },  
    "auComponentId": { "type": "exact", "field": "auComponentId" },  
    "aaxId": { "type": "exact", "field": "aaxId" },  
    "vendorFuzzy": {  
      "type": "fuzzy",  
      "fields": ["vendor", "name"],  
      "threshold": 0.85  
    },  
    "parameterSignature": {  
      "type": "ml",  
      "description": "ML-based matching on parameter structure",  
      "status": "planned"  
    }  
  }  
}
```

— 7.3 PARAMETRI NASCOSTI E DISCOVERY

[NOTE] Alcuni plugin espongono parametri interni o nascosti solo tramite scan esplicito. WhyCremisi esegue una scansione completa dei parametri alla prima connessione e li confronta con il database. Parametri non documentati vengono aggiunti con flag "discovered": true per review manuale.

```
{
  "id": "unmapped_param_47",
  "name": "Internal Param 47",
  "index": 47,
  "type": "float",
  "discovered": true,
  "needsReview": true,
  "notes": "Scoperto durante scan automatico — parametro non documentato nel manuale ufficiale"
}
```

• 8. Esempi Concreti

— 8.1 FABFILTER PRO-Q 3 — 24 BANDE, 5 TIPI DI FILTRO, SPECTRUM

```
{
  "id": "fabfilter_proq3",
  "name": "Pro-Q 3",
  "vendor": "FabFilter",
  "category": "ParametricEQ",
  "parameters": [
    { "id": "band1_freq", "index": 0, "type": "float", "min": 20, "max": 20000, "unit": "Hz", "default": 1000 },
    { "id": "band1_gain", "index": 1, "type": "float", "min": -30, "max": 30, "unit": "dB", "defaultValue": 0 },
    { "id": "band1_q", "index": 2, "type": "float", "min": 0.025, "max": 40, "defaultValue": 0.707 },
    { "id": "band1_type", "index": 3, "type": "enum", "values": ["Peak", "LowShelf", "HighShelf", "LowPass", "HighPass"], "defaultValue": "Peak" },
    { "id": "band1_enabled", "index": 4, "type": "bool", "defaultValue": true }
  ],
  "parameterCount": 200,
  "presetCount": 25,
  "knownProcedures": ["vocal_cleanup", "bass_tighten", "mastering_eq"]
}
```

— 8.2 IZOTOPE OZONE 10 — 8 MODULI, MASTER ASSISTANT

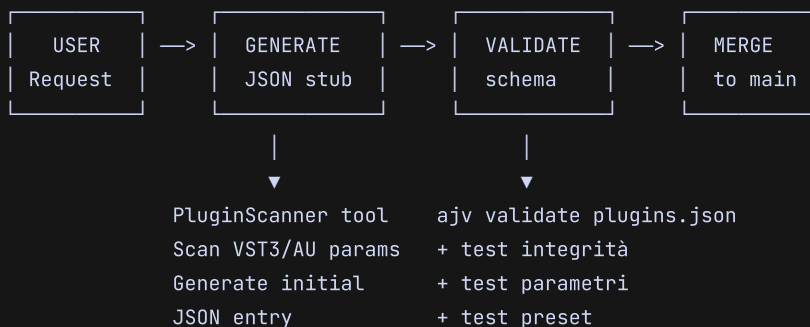
```
{
  "id": "izotope_ozone10",
  "name": "Ozone 10",
  "vendor": "iZotope",
  "category": "LoudnessMaximizer",
  "modules": [
    { "name": "EQ", "enabled": true, "parameters": ["eq_band1_freq", "eq_band1_gain", "eq_band1_q"] },
    { "name": "Dynamics", "enabled": true, "parameters": ["dynamics_threshold", "dynamics_ratio", "dynamics_attack", "dynamics_release"] },
    { "name": "Imager", "enabled": true, "parameters": ["imager_width", "imager_stereo_balance"] },
    { "name": "Maximizer", "enabled": true, "parameters": ["maximizer_threshold", "maximizer_irc_mode"] },
    { "name": "Match EQ", "enabled": false },
    { "name": "Spectral Shaper", "enabled": false },
    { "name": "Vintage Tape", "enabled": false },
    { "name": "LowEnd Focus", "enabled": false }
  ],
  "masterAssistant": {
    "available": true,
    "params": ["target_loudness", "target_dynamics", "style"]
  },
  "parameterCount": 180
}
```

```
{
  "id": "valhalla_valhallaroom",
  "name": "ValhallaRoom",
  "vendor": "ValhallaDSP",
  "category": "Reverb",
  "algorithms": ["Nano", "Warm", "Smooth", "Quiet", "Bright", "Concrete", "Chamber", "Hall", "Plate",
  "parameters": [
    { "id": "mix", "index": 0, "type": "float", "min": 0, "max": 100, "unit": "%" },
    { "id": "decay", "index": 1, "type": "float", "min": 0.01, "max": 100.0, "unit": "s" },
    { "id": "pre_delay", "index": 2, "type": "float", "min": 0, "max": 500, "unit": "ms" },
    { "id": "size", "index": 3, "type": "float", "min": 0.01, "max": 4.0 },
    { "id": "diffusion", "index": 4, "type": "float", "min": 0, "max": 100 },
    { "id": "modulation_rate", "index": 5, "type": "float", "min": 0, "max": 20, "unit": "Hz" },
    { "id": "modulation_depth", "index": 6, "type": "float", "min": 0, "max": 100 },
    { "id": "high_cut", "index": 7, "type": "float", "min": 100, "max": 20000, "unit": "Hz" },
    { "id": "low_cut", "index": 8, "type": "float", "min": 20, "max": 2000, "unit": "Hz" },
    { "id": "algorithm", "index": 9, "type": "enum", "values": ["Nano", "Warm", "Smooth", "Quiet", "Br
  ],
  "parameterCount": 40,
  "presetCount": 128
}
```

— 8.4 XFER SERUM — WAVETABLE OSCILLATOR, 45 PARAMETRI, 200+ PRESET

```
{
  "id": "xfer_serum",
  "name": "Serum",
  "vendor": "Xfer Records",
  "category": "Synth",
  "oscillators": [
    { "id": "oscA", "type": "wavetable", "parameters": ["osc_a_wavetable", "osc_a_coarse", "osc_a_fine"
    { "id": "oscB", "type": "wavetable", "parameters": ["osc_b_wavetable", "osc_b_coarse", "osc_b_fine"
    { "id": "sub", "type": "sub", "parameters": ["sub_level", "sub_waveform"] },
    { "id": "noise", "type": "noise", "parameters": ["noise_level", "noise_sample"] }
  ],
  "filters": [
    { "id": "filter1", "type": "multimode", "parameters": ["filter1_type", "filter1_cutoff", "filter1
  ],
  "modulation": {
    "envelopes": 4,
    "lfo": 4,
    "matrixSlots": 16
  },
  "parameterCount": 200,
  "presetCount": 450
}
```

— 9.1 WORKFLOW PER AGGIUNGERE UN PLUGIN



— 9.2 COMMUNITY CONTRIBUTION MODEL

```
{
  "contributionModel": {
    "type": "crowd-sourced",
    "platform": "GitHub PR",
    "requirements": [
      "Validazione JSON Schema superata",
      "Plugin posseduto e testato dal contributor",
      "Almeno 80% parametri mappati",
      "5 preset funzionanti minimo"
    ],
    "reviewProcess": [
      "Automatica: validazione CI + test integrità",
      "Manuale: verifica parametri da maintainer",
      "Beta: plugin marcato come 'community' per 30 giorni",
      "Stable: promosso dopo conferma utenti"
    ],
    "scoreSystem": {
      "baseContribution": 10,
      "perParameter": 1,
      "perPreset": 3,
      "perProcedure": 5,
      "verificationBonus": 20
    }
  }
}
```

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "PluginDatabase",
  "type": "object",
  "required": ["schemaVersion", "plugins"],
  "properties": {
    "schemaVersion": { "type": "string", "pattern": "^\\d+\\.\\.\\d+$" },
    "plugins": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/definitions/PluginEntry" }
    }
  },
  "definitions": {
    "PluginEntry": {
      "type": "object",
      "required": ["id", "name", "vendor", "category", "parameters"],
      "properties": {
        "id": { "type": "string", "pattern": "^[a-z][a-z0-9_]+$" },
        "name": { "type": "string" },
        "vendor": { "type": "string" },
        "category": { "type": "string", "enum": ["ParametricEQ", "DynamicEQ", "Compressor", "Limiter"] },
        "parameters": {
          "type": "array",
          "items": { "$ref": "#/definitions/ParameterEntry" }
        }
      }
    },
    "ParameterEntry": {
      "type": "object",
      "required": ["id", "name", "index", "type"],
      "properties": {
        "id": { "type": "string" },
        "name": { "type": "string" },
        "index": { "type": "integer", "minimum": 0 },
        "type": { "type": "string", "enum": ["float", "int", "bool", "enum", "text"] },
        "min": { "type": "number" },
        "max": { "type": "number" },
        "defaultValue": {}
      }
    }
  }
}
```

— 9.4 TEST DI INTEGRITÀ

TEST	DESCRIZIONE	COMANDO
JSON Valido	Parsing JSON corretto	<code>ajv validate -s schema.json -d plugins.json</code>
GUID Unici	Nessun duplicato GUID	<code>`jq '[.plugins[]].vst3Guid]`</code>
Parametri Raggiungibili	Indici contigui 0..n	Script Python custom
Preset Validati	Parametri preset esistono nel plugin	<code>node tests/validate-presets.js</code>

[NOTE] Il database viene rigenerato automaticamente da scansione ogni volta che un nuovo plugin viene rilevato sul sistema dell'utente. La scansione produce un "stub JSON" che il contributor può completare con preset, procedure e metadata.

• **Appendice A: Riepilogo Struttura File**

```
plugins.json
├─ schemaVersion: "2.0"
├─ lastUpdated: "2026-05-12"
├─ pluginCount: 100
├─ plugins: PluginEntry[100]
│   └─ [0]: id, name, vendor, version, category, formats
│       └─ parameters: ParameterEntry[200] max
│           └─ presets: PresetEntry[300] max
│               └─ knownProcedures: ProcedureEntry[50] max
│                   └─ metadata: tags, urls, versions
│   └─ [1]: ...
│   └─ [99]: ...
├─ categories: { 50+ categorie definite }
└─ $schema: "./schema/plugin-database.schema.json"
```

• **Appendice B: Metriche di Database Stimate (100 Plugin)**

METRICA	VALORE
Plugin totali	100
Parametri totali	~8,500
Media parametri/plugin	85
Preset factory totali	~4,500
Procedure totali	~300
Categorie	54
GUID VST3	100
Dimensione JSON (grezzo)	~5 MB
Dimensione JSON (gzip)	~600 KB
Tempo validazione ajv	< 50ms